

Real-Time Operating Systems Course

Course Presentation

Paulo Pedreiras

DETI/UA/IT

September 11, 2026



Faculty

- Paulo Pedreiras (Course Coordinator)
- pbrp@ua.pt
- 4.3.21 (DETI) or 19.1.23 (IT)

What are real-time systems?

Preliminaries

What are real-time systems?

- Computational systems
- Subject to the evolution of real-time (or physical time, if you wish)

Systems in which it is required to do the right thing at the right time, otherwise things can go (very) wrong!



Course objectives

- Main topic:
 - **Operating Systems** , **Programming Techniques** and **Analysis tools** to support the design of systems that interact with (or simulate a) physical process (or environment) so that they can do the **right thing at the right time**
- We will address:
 - The source and characterization of the restrictions imposed by the environment to the temporal behavior of the computational system;
 - Approaches to allow the computational system to be aware of the state of the environment that surrounds it;
 - The scheduling theory of concurrent tasks associated with real-time processes;
 - The structure and functionality of real-time operating systems
 - **And apply all these knowledge in practice!**

Examples of Real-Time Systems

Agile Manufacturing



Traffic control system

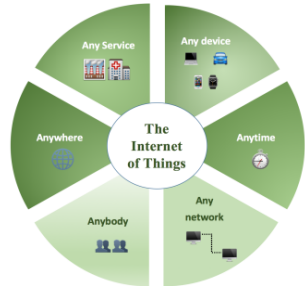
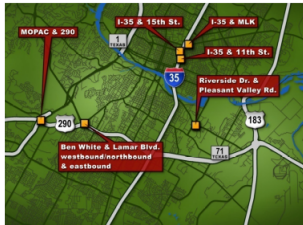


Image Credits: Realtime Robotics

Common misunderstandings

- Isn't a quick processor sufficient?

Common misunderstandings

- **Isn't a quick processor sufficient?**
 - For a simple program, with a single task, that may work. However, when CPU(s) have to execute several tasks concurrently, just a quick processor may not be enough! Some tasks may block others and cause big and/or unpredictable delays.
- **Then, what do we need?**

Common misunderstandings

- **Isn't a quick processor sufficient?**

- For a simple program, with a single task, that may work. However, when CPU(s) have to execute several tasks concurrently, just a quick processor may not be enough! Some tasks may block others and cause big and/or unpredictable delays.

- **Then, what do we need?**

- Scheduling! Which means, select the right task to execute in every instant.
There are scheduling algorithms that allow predicting and minimizing the maximum delays that tasks may suffer.

Common misunderstandings

- **So all this stuff applies only when we need multi-tasking... ?**

- As stated above, we are considering situations where a computer has to perform several tasks simultaneously. It is normal that in such situations we use one multi-tasking operating system/kernel.
- But often, even when the main body of the program is a simple endless loop, there are various pseudo-tasks, part of asynchronous interrupt routines, which lead to the same situation.

The triggering of the interrupt routines may also be delayed, or even discarded. We must use proper techniques to bound and compute these delays.

Common misunderstandings

- **Are those delays so important?**

- It depends on the kind of system. If we're talking e.g. about control systems on a plane, a car's X-by-wire or a robot that moves around other people and equipment ... **THEN YES!**
- On the other hand, if we are talking about multimedia systems or routers in networks, delays in tasks shall not cause death to anyone, but there will be a loss of Quality-of-Service.



Course organization

- One TP class (3 H per week) organized in two parts:
 - **Theoretical part:** presentation and discussion of concepts, models, techniques, algorithms, etc.
 - Students should read selected parts of the base bibliography
 - Slides are available at the course website a few days before the class
 - Discussions are welcome!
 - **Practical/Lab part:** learning about RTOS and tools, with focus on its practical use
 - Groups of 2 students
 - Two RTOS: Linux RT services and Zephyr
 - Tutorial activities (first half of the semester) followed by a project

Assessment

Normal period

- Theoretical part: 50%
 - Written exam
- Practical part: 50%:
 - Practical exam: in-class, group, lab PCs, Linux RT-Services (50%)
 - Project: in-class demo and oral defense, group, Zephyr + micro-controller (50%)

Resit period

- Theoretical component: 50%
 - Written exam.
- Practical component: 50%
 - Practical mark from the normal period or lab exam.
 - Lab exam requires carrying out the theoretical written exam also during the resit period.

Approval

- Approval requires earning a minimum grade of 8.0 (0-20 scale) on the written exam / theoretical part (UA, Regulamento n.º 395/2026, art. 29, 4 a) and a final score ≥ 9.5 .



- **No AI:** No AI can be used for the written and practical exams. AI cannot be used also during the demonstration and oral defense of the final project.
- **Constrained AI:** AI can and should be used for the development of non core aspects of the practical works and project. These include e.g. unit testing, auxiliary functions not related with the main topics (e.g. a queue management module, a sorting algorithm).
- **Additional uses of AI:** AI can be effectively used for self-study as a complement to the CU bibliography (books, handouts, exercise books). There are no restrictions on the use of AI in this context, but it should be used with care because AI-generated answers are not guaranteed to be accurate.

Collaboration policy: cheating and plagiarism (1/4)

As MSc students these rules should be familiar to you, but anyway

...

- **Collaboration among students is** not only allowed but **encouraged**, even if they don't belong to the same work group. It allows to build team spirit and understand the benefits of team work.
- However, **the academic credit awarded to a student or work group should represent the own work of the student or work group**. Therefore, students are expected to produce their own original academic work, so there are natural limits to the cooperation.

What is strictly forbidden:

- For lab works and lab project:
 - Sharing code snippets (modified or otherwise) with other teams;
 - Share any sort of solution to project-specific problems (e.g. algorithms) with other teams;
 - Using solutions from prior years.
- Written exams:
 - The activities are strictly individual. Any sort of collaboration during its realization is considered cheating.

Collaboration policy: cheating and plagiarism (3/4)

- Algorithms and/or code snippets adapted from external sources should be clearly identified in the code and report. The “golden rule” is that **either the code is developed by the student/group or it is clearly credited to the original authors**. Failing to comply with this rule is plagiarism.
- Materials submitted by students are compared against code and reports submitted by other students (even in prior years, when suitable). Tools such as moss (<https://theory.stanford.edu/~aiken/moss/>) and Turnitin (<https://www.ua.pt/pt/inovacaopedagogica/turnitin>) are systematically applied.

Collaboration policy: cheating and plagiarism (4/4)

Note also that **students that facilitate cheating** (e.g. by sharing its own code with others) **are jointly responsible** for the infraction.

Students involved on a cheating or plagiarism situation are subject to the following consequences:

- 0 (zero) mark on the activity (lab work, lab project, quiz or exam)
- Elaboration of a disciplinary action procedure

Course Schedule



Available, and eventually updated, in the CU site at UA's eLearning.

Includes:

- Class topics
- Recommended readings

Click to access Course Schedule <https://elearning.ua.pt/mod/resource/view.php?id=117051>

Main bibliography

Main bibliography. Additional references are provided in the course materials, where and when appropriate.

- Base
 - Giorgio Buttazzo (2011). "HARD REAL-TIME COMPUTING SYSTEMS: Predictable Scheduling Algorithms and Applications", Third Edition, Springer, 2011. (available at the course site, educational use only)
- Complementary
 - Peter Marwedel (2021), "Embedded System Design-Embedded Systems Foundations of Cyber-Physical Systems, and the Internet of Things", Springer, ISBN 978-3-030-60910-8
 - Sanjoy Baruah, Marko Bertogna, Giorgio Buttazzo (2015), "Multiprocessor Scheduling for Real-Time Systems (Embedded Systems)", Springer, Available at <https://www.springer.com/gp/book/9783319086958>
 - Andrew Elias (2024), "Zephyr RTOS Embedded C Programming: Using Embedded RTOS POSIX API", 1st Edition, Apress, ISBN-10: 886880106H, ISBN-13: 979-8868801068